Automatic Evasion of Machine Learning-Based Network Intrusion Detection Systems

Haonan Yan, Xiaoguang Li, Wenjing Zhang, Rui Wang, Hui Li, *Member, IEEE*, Xingwen Zhao, Fenghua Li, and Xiaodong Lin, *Fellow, IEEE*

***Abstract*—Network intrusion detection systems (IDS) are often considered effective to thwart cyber attacks. Currently, state-of-the-art (SOTA) IDSs are mainly based on machine learning (ML) including deep learning (DL) models, which suffer from their own security issues, especially evasion attacks by using adversarial examples. However, previous studies mostly focus on extracted features rather than the traffic sample itself, and/or assume that the adversary knows the information of the target model more or less, which severely restricts attack feasibility in practice. In this paper, we re-investigate this problem in a more realistic label-only black-box scenario and propose a practical evasion attack strategy to solve the above limitations. In this newly considered case that the adversary morphs the traffic sample and only obtains the results accepted or rejected without other knowledge, we successfully leverage the model extraction and transfer attack to evade the detection. The entire attack strategy is automated and a comprehensive evaluation is performed. Final results show that the proposed strategy effectively evades seven typical ML-based IDSs and one SOTA DL-based IDS with an average success rate of over 75%. We also discuss the corresponding countermeasures against our attack, which finally highlight the need for effective defenses against our attack.**

***Index Terms*—Adversarial traffic example, black-box evasion attack, model extraction attack, network intrusion detection system, transfer attack.**

I. Introduction

NETWORK intrusion detection systems (IDS) are utilized as a common and effective measure to classify malicious traffics in advance to resist cyber-attacks which cause serious issues such as equipment halt and privacy breach [1], [2]. There are two categories of IDS, signature-based and model-based IDSs. The traditional signature-based IDSs can no longer meet the increasingly complex and ever-changing network detection requirements. Meanwhile, the excellent performance of machine learning (ML), especially deep learning (DL) on classification tasks allows it to be widely used as the core classifier of the IDS to detect new and malicious traffic for network security. Admittedly, ML-based IDS is state-of-the-art in network security detection [3], [4]. As a result, the ML-based IDS is considered in this work.

Despite the fact that ML models are essential for the IDS, they lack reliability because the malicious sample may be misclassified as accepted, resulting in false-negative outcomes [5]. This causes the attack to go undetected even though it actually occurred. Besides, the emergence of adversarial examples also exacerbates the detection error problem [6]. Motivated adversaries can add trivial perturbations into the traffic sample to make it be misclassified by the target model, leading to an evasion attack, which further shows the vulnerability of the ML-based IDS[7].

The evasion attack in ML models can be divided into two categories: white-box attacks and black-box attacks, depending on the information about the target model mastered by the adversary. For white-box attacks, it assumes that the adversary knows all relevant information about the target IDS, such as classification algorithms, model internals, and training dataset. Some representative works [8], [9], [10] successfully implement evasion attacks, but it is impossible for an adversary to access all these information, so such an assumption is not reasonable in practice. For black-box attacks, knowing the classification scores of the output results, Xu et al. [11] automatically evade the classifier by black-box accessing the target model. The adversary in these works needs to know the information of the target model more or less. However, IDS is a typical label-only black-box system, it only outputs abnormal (attack detected) or normal (no attack detected), and the confidence score is not output to the adversary. This means that the only information the adversary can obtain is whether the detection was successfully evaded or not, without confidence scores. This practical situation makes existing related attacks infeasible in IDS cases. To our best knowledge, (*Limitation 1*:) *none of the existing evasion attacks against ML-based IDSs considers feasibility in a real-world label-only black-box scenario*.

Manuscript received 4 June 2022; revised 7 February 2023; accepted 16 February 2023. Date of publication 22 February 2023; date of current version 12 January 2024. The work of Hui Li was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3103400, in part by Shaanxi innovation team project under Grant 2018TD-007, and in part by Higher Education Discipline Innovation 111 project under Grant B16037. The work of Haonan Yan was done when he visits the School of Computer Science at the University of Guelph. *(Corresponding authors: Xiaoguang Li; Hui Li.)*

Haonan Yan is with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi'an 710126, China, and also with the School of Computer Science, University of Guelph, Guelph, ON N1G 2W1, Canada (e-mail: yanhaonan.sec@gmail.com).

Xiaoguang Li, Rui Wang, Hui Li, and Xingwen Zhao are with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi'an 710126, China (e-mail: xg_li@outlook.com; 785340571@qq.com; lihui@mail.xidian.edu.cn; xwzhao@xidian.edu.cn).

Wenjing Zhang and Xiaodong Lin are with the School of Computer Science, University of Guelph, Guelph, ON N1G 2W1, Canada (e-mail: wzhang25@uoguelph.ca; xlin08@uoguelph.ca).

Fenghua Li is with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academic of Sciences, Beijing 100045, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: lfh@iie.ac.cn).

Digital Object Identifier 10.1109/TDSC.2023.3247585

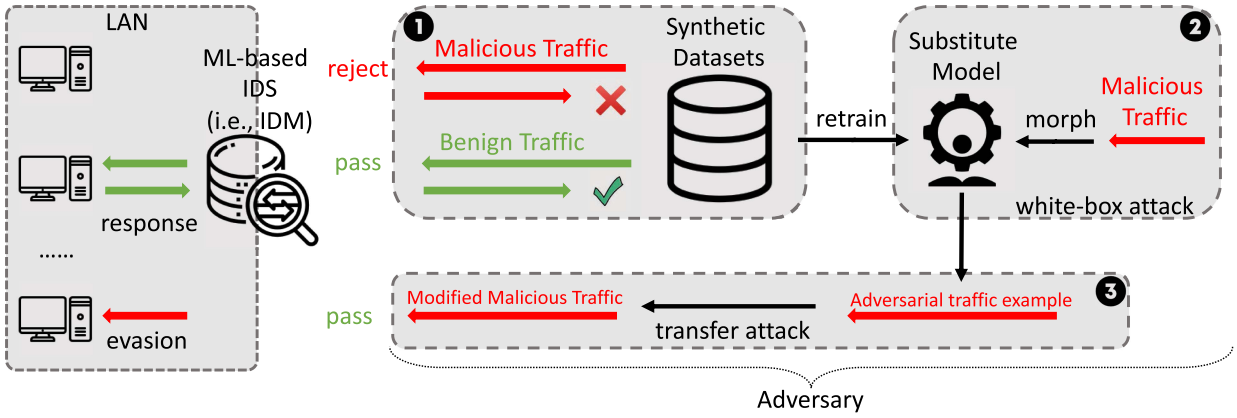


Fig. 1. An illustration of our evasion attack strategy. The procedure consists of three steps, i.e., step 1: model extraction, step 2: local white-box attack, and step 3: black-box transfer attack.

Although there are many related evasion attacks launched around images [12], text [13], and malicious files[14], but these works are not suitable for traffic scenarios. This is because traffic samples follow a specific protocol format, and these works cannot ensure the functionality of traffic after modification. Most previous related works modify the traffic feature, but in practice adversaries often need to directly provide adversarial traffic samples, which further limits the capability of adversaries. Another reason is that, unlike images that inherently contain explicit features that can be directly inputted into the model, traffic features are statistical and thus cannot be modified directly. Moreover, the obfuscation scheme in [15] does not explore adversarial traffic samples to optimize their attack effects and costs. Consequently, (*Limitation 2*:) *there is still a lack of a practical adversarial traffic sample construction method.*

In this work, we propose a practical automatic black-box evasion attack against the network ML-based IDS. Compared with previous works, we consider a more realistic evasion scenario, where the adversary does not have any knowledge of the target ML-based IDS, and only the output of the target ML-based IDS is available, i.e., the adversary's ability is consistent with that of normal users. Our attack strategy is as follows:

Step 1. *a model extraction attack* is performed on the target ML-based IDS to obtain local substitute models, which allow adversaries to apply previous adversarial example methods with full access to the model. For enhancing extraction efficiency, three synthetic queries are devised in traffic scenarios to extract the decision boundary of the target ML-based IDS. Besides, since a single model is difficult to deal with complex scenarios, ensemble learning is introduced to train the local substitute model with lower number of queries and higher data utilization.

Step 2. *a local white-box attack* is performed on the extracted models to craft adversarial traffic examples. Considering statistical features of traffic, three morphing schemes for adversarial traffic examples are devised to replace the current common scheme of adding noise directly, which cannot maintain the original maliciousness and is not feasible in the traffic domain. The adversary could also formulate the attack into an optimization problem to guide the morphing direction.

Step 3. *a black-box transfer attack* is performed by using adversarial traffic samples. Due to the transferability of adversarial examples [16], malicious traffic examples can evade the detection of the target ML-based IDS with a high success rate.

We implement the framework which automates the complete attack process. Our evasion process is shown in Fig. 1. The framework improves the two limitations aforementioned. We also experimentally evaluate the framework and the corresponding defenses.

Contribution. The main contributions are summarized as follows:

1. *Devise a practical evasion attack strategy against ML-based IDS.* We propose an automatic evasion strategy for ML-based IDS in a realistic label-only black-box scenario, combining model extraction and transfer attack, and improve the extraction attack for traffic scenarios, which is more practical and effective than previous adversarial attacks for network traffic.
2. *Explore the generation method of evasive traffic samples.* We provide practical adversarial traffic example morphing methods under the constraint of network protocol format, without any knowledge about the target model's internal states. Our method guarantees the evasion effectiveness and protocol functionality of the sample. We also formulate the construction of evasive samples as an optimization problem to guide the sample morphing and reduce the attack cost.
3. *Evaluate our attack and defense.* We implement and evaluate our ML-based IDS evasion framework which automates all attacks and improvements aforementioned to test the robustness of ML-based IDS. Experiments demonstrate that our attack is effective on seven kinds of typical ML-based models and one state-of-the-art DL-based model. We also discuss corresponding countermeasures.

The final results show that they have a certain effect in defending our attacks, but each of them has great limitations, which highlights the need for effective defense against our attacks.

Roadmap. The rest of the paper is organized as follows: Section II describes the motivating scenario and its challenges; Section III elaborates on our attack strategy, including the process of extraction and transfer attack; Section IV presents our measurement study and experiment results; Section V introduces three directions of defense against our proposed attack; Section VI discusses the limitations of our current design and potential future research; Section VII reviews related prior research and Section VIII concludes the paper.

II. Problem Description

In this section, we introduce the attack scenario to illustrate the problem of ML-based IDS in Section II-A. Then, we present our evasion idea in Section II-B and the corresponding challenges in Section II-C.

A. Motivating Scenario

Administrators often set up the IDS at network entrances to detect and prevent malicious traffic timely. At present, the most advanced detector usually takes a machine learning model as a core classifier, which is pre-trained by extracting features from a collection of typical malicious traffic as learning samples.

Regular malicious traffic is easily and effectively detected by the existing ML-based IDSs [17]. Therefore, adversaries try to evade the detection of the target ML-based IDS by adaptively modifying the features of the malicious traffic. They can deliberately construct traffic samples to request the detector and observe the binary response from the detector, which is also called a label-only black-box scenario.

However, this brings the new problem that frequent malicious probing with a single account or IP will cause the administrator to be alert and take corresponding countermeasures such as banning requests or updating the classification model. The work in [18] shows that it is possible to identify the purposeful queries for adversarial examples from the past queries. Besides, a large number of queries increase the risk of being detected. Therefore, the adversary needs to construct effective malicious traffic within a limited number of tests. There are two cases white-box and black-box depending on whether the adversary knows the internal information of the IDS. Note that the white-box scenario has limited applicability in practice for IDS. Here, we consider the case where the adversary relies on the least amount of information about the target, which is the most difficult scenario to devise evasion strategies, i.e., the adversary has no knowledge about the internal condition of the target detector, such as model algorithm, training datasets, or feature space.

B. Evasion Idea

In order to construct the malicious traffic example evading detection effectively, adversaries can utilize its transferability. Specifically, they perform the extraction attack and test the malicious sample on the obtained local model. Although adversaries may not know the traffic features used in the target model, changes in the traffic samples can lead to changes in the features extracted by the target ML-based IDS. On the basis of this, adversaries can constantly morph the traffic sample to change its feature value while ensuring samples' maliciousness, and thus obtaining a series of variants. These traffic variants misclassified by the local model are expected to evade the detection of the target ML-based IDS since the local model is generated from the ML-based IDS. As a result, the adversary can successfully evade the detection of the IDS and current ML-based IDSs are under the threat of this type of new evasion attack.

C. Challenges in Black-Box Evasion

When the constructed sample is misclassified by the local substitute model, adversaries expect that sample to evade the detection due to the transferability between the machine learning models. Two important challenges emerge here.

The first challenge is how to make traffic samples more transferable. The transferability works better when the extracted model is high-fidelity to the original model, and two models should be functionally consistent, i.e., give the same response to the same request. However, with black-box access, the adversary can only get a binary output of acceptance/rejection from the detector, which greatly increases the difficulty of extraction. Most of the previous works contain some auxiliary information more or less, such as confidence values or model algorithms. Nevertheless, in practice, this situation does not occur often.

The second challenge is how to provide valid malicious traffic samples. One malicious sample is easily morphed to pass the detection, but it is difficult to ensure that the maliciousness of the modified packet remains. The essence of successful evasion is that the adversary achieves misclassification by finding a blind spot or ambiguous decision boundary of the target classifier, where malicious samples are treated as benign samples. In the network scenario with a limited number of accesses, previous works [12], [13], [14] are not feasible due to the specificity of the traffic sample under the specific protocol format.

III. Proposed Auto Evasion Framework

In this section, we propose a framework for generating malicious adversarial traffic samples to automatically evade ML-based IDS detection. First, we present synthetic queries used to extract the ML-based IDS in Section III-A and the algorithm selection for local substitute model training in Section III-B. Then, we devise adversarial traffic examples using this substitute model in Section III-C. Finally, we provide an optimization problem to formulate the adversarial attack. The framework automates our whole attack strategy, which is more practical than previous works, as illustrated in Fig. 2.

As stated in Section II-A, the main strategy is to train local substitute models for the target ML-based IDS using synthetic datasets in the black box scenario first. The labels come from the output of the target ML-based IDS. When adversarial traffic examples are crafted using the substitute model, these examples

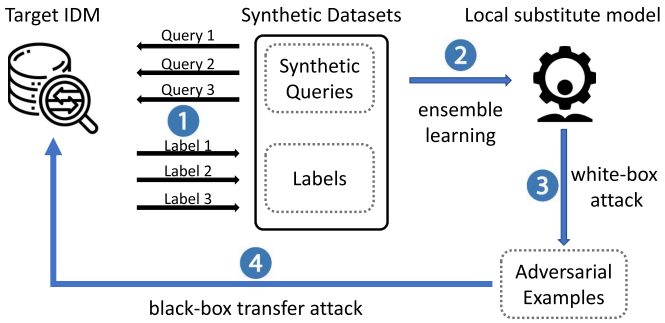


Fig. 2. Illustration of the proposed auto evasion framework.

are expected to evade ML-based IDS detection with a high success rate due to their transferability. The works [16], [19] also prove the transferability of adversarial examples even among the heterogeneous models.

A. Constructing Synthetic Queries

Adversaries need lots of labeled datasets to train the local substitute model. The reason is that machine learning models with good performance always require large data for training. However, blind queries will increase the cost to adversaries, and alert administrators after sending a large number of ineffective malicious queries. Therefore, the most common method for adversaries currently is to synthesize queries and treat the target ML-based IDS as an oracle to label these queries. With this synthetic dataset, adversaries could train the local substitute model of the target ML-based IDS. The previous representative data synthesis method is Jacobian-based dataset augmentation [16]. It is essentially a reference to the idea of gradient descent. By using this method, new sample points are obtained along with the sign of the Jacobian matrix dimension based on the initial point, which essentially follows the principle of gradient descent. However, this method is not suitable for traffic because it is limited by the selection of initial points, and it is not stable. Moreover, the relationships among traffic features are complex and the value of features varies greatly, which makes the calculation of the Jacobian matrix more difficult.

Conversely, we propose three practical query construction methods here. The first two kinds of queries are in the feature space, the adversary could manipulate the packet to adjust its feature to the query one, details are shown below. Formally, for the traffic example t to be queried, its extracted traffic feature vector is $\boldsymbol{x}$, and thus we have

$$\boldsymbol{x} = \mathrm{E}(t), \tag{1}$$

where E denotes the feature extraction function. Although adversaries have no knowledge of features in the target model under our setting and different ML-based IDSs may use different features in the training, we observe that most features in traffic classification come from three main categories, and these features are highly correlated. Based on our statistics, here we summarize and select the features used frequently in previous works shown in Table II, and construct query traffic examples using these features. Note that our scheme is a packet-level scheme, even if unseen ML-based IDS uses new or different features, these features will also change with the packet change, which means our scheme also works.

1) Query 1: Linear Query Matrix: We build a linear feature query matrix $\boldsymbol{M}$ composed of an identity matrix and a zero-row vector, shown as

$$\boldsymbol{M} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \\ 0 & 0 & \cdots & 0 \end{bmatrix}_{(d+1)\times d},$$

where d is the number of the common feature dimension. The element 1 in i-th ($i = 1, 2, \ldots, d$) row of $\boldsymbol{M}$ is used to solve for the i-th weight coefficient of the target linear model. The last row in $\boldsymbol{M}$ is used to solve the bias of the model. The linear query matrix can also be denoted as

$$\boldsymbol{M} = \begin{bmatrix} \boldsymbol{m_1} & \cdots & \boldsymbol{m_d} & \boldsymbol{0} \end{bmatrix}^{\top}.$$

For the construction of traffic example t, it is impossible to directly construct traffic examples with extracted feature values of 0 or 1, but we can construct examples whose features are the minimize and maximize value to respectively approximate the 0 and 1. Once the feature dimension i is determined, we can empirically adjust the query traffic example t in (1) to let $\boldsymbol{x_i}$ obtain the extreme value and other feature dimensions keep the same. For example, when we try to extract the coefficients of the packet length in the target model, we can make the constructed traffic example length 40 or 1480 bytes, and the feature value after feature extraction is the minimum or maximum, which is approximately 0 or 1 in the linear query matrix.

The principle of the linear query matrix is solving equations by linear algebra. It can extract the parameters of the target model with the least theoretical cost and 100% successful rate, i.e., it copies the target model exactly. It can run quite fast as it only needs m+1 queries and calculations. The only weakness is that it only works on linear models.

2) Query 2: Feature Space Traversal: Considering real-world scenarios, the adversary can prejudge the value range of each feature of the traffic sample. For example, the packet length of each flow ranges from 40 to 1480 bytes. Thus, the adversary can construct samples by traversing each feature space according to the arithmetic sequence, then shuffling samples in each feature column, and concatenating all feature columns into the synthetic dataset finally. Formally, suppose D is the value range of one sample's feature, and the size of the synthetic dataset is n, the traversal interval ρ should be

$$\rho = D/(n-1).$$

After the feature value is determined, the traffic example can be constructed by the adversary's manipulation. Note that many features are interconnected, for example, the length of the payload and the length of the packet, it is impossible to increase the payload but decrease the packet's length. Therefore, we divide

features into three categories from three independent dimensions, i.e., length-related features, the number of packets, and time-related features, and only traverse representative features of these three dimensions in our experiments. The construction method of the traffic sample is the same as the one in Query 1.

The ML-based IDS's "knowledge", i.e., its malicious traffic detection capability, is derived from the training dataset. Thus, the training dataset of one effective ML-based IDS should comprehensively cover the value space of the feature as much as possible. This type of query can ensure that the ML-based IDS's training dataset is always a subset of the synthetic dataset so that the local substitute model trained with the synthetic dataset will have high fidelity to the target model.

3) Query 3: Dataset Migration: In this work, we use typical traffic datasets as the basement queries. The representative dataset selection method is also applied to reduce the number of queries for the target model extraction. Excellent ML-based IDSs have the basic competency requirement that performs well on typical attack traffic datasets (e.g., CIC-IDS2017 [20]). The traffic features of the same kind of network attack are basically identical, even in different datasets. Besides, the training dataset of one excellent ML-based IDS always contains many normal attack data, which are similar to these typical attack datasets. Even if the target model never has access to these typical datasets, these datasets can still help determine the decision boundary of the target model.

After queries are sent to the target model, adversaries use the accepted or rejected feedback as labels to construct the training dataset for the local substitute model. The adversary constructs the query dataset using the 3 synthetic query methods aforementioned. These queries are generated considering simplicity and efficiency, and the adversary can flexibly adjust the proportion of three queries to request the target model simultaneously depending on the scenario.

B. Extracting Detection Model

Constructing malicious examples with full knowledge of the model parameters can significantly improve the adversarial properties of examples. Therefore, it is helpful to construct a local substitute model pertinently. Note that the ultimate goal of the model extraction is to train a substitute model approximating the decision boundary of the target model rather than a more accurate model.

1) Algorithm Selection: In black-box scenarios, adversaries have no knowledge about the model algorithm of the target ML-based IDS. Nevertheless, many existing studies, such as knowledge distillation [21], have shown that substitute models can have high fidelity for the complex structured source models. For instance, one complex deep neural network model can be functionally replaced by a well-structured logistic regression or decision tree model [22]. Therefore, we run several different machine learning algorithms, as shown in experiments. After a large range of experiments, the final substitute models can reach more than 80% fidelity of the target model, which is enough to perform subsequent attacks.

2) Local Model Training: The proper choice of local training model can improve data utilization. The basic idea to select the algorithm of the substitute model is by utilizing the cross-validation method. The model with the best performance is chosen for a later test. To further improve the consistency of the local model, we introduce ensemble learning and implement it with mature autoML technology [23]. The reason is that different models usually have different transfer rates for the same target model. One single model is difficult to cope with different target models and its transfer rate is low. Besides, using synthetic datasets from the target model, ensemble learning makes the extracted decision boundary be more towards the target model. The work [24] also shows that using an ensemble of local models can improve the transfer rate of adversarial examples.

In our framework, the adversary can automatically choose the current best model combination, feature preprocessing steps, and also set their respective hyper-parameters, according to their performance on the synthetic dataset. Local models based on ensemble learning yields the highest transfer rate in our experiment, which actively demonstrates that local ensemble can steal the decision boundaries of the target model to the maximum extent possible, nearly the same decision on the new input as the oracle. We also validate the performance of ensemble models and compare the transfer rate in Section IV-B.

C. Devising Evasive Traffic Example

ML-based traffic classification schemes usually use the statistical features of traffic sessions as the classification basis. The basic unit, session, is composed of all packets in one complete connection. After statistical analysis, we found that network traffic features mainly come from three dimensions: length, number, and time. Though adversaries do not know the features used in target ML-based IDS, malicious tampering can be performed during the transformation from traffic to features by modifying the packet length/number and traffic duration. Furthermore, as long as the adversary ensures that modified packets still conform to the original protocol standard, the functionality of modified packets will not be compromised. Based on this, we provide three methods to modify the extracted features by morphing the malicious traffic sample to evade detection.

1) Methods of Morphing Traffic: Network flows are first preprocessed into features, and then these features are used by machine learning models to classify the traffic. Therefore, malicious tampering can be performed during the transformation from network flow to features. Currently, most network features are extracted mainly from three directions: the length of packets, the number of packets, and the lasting of the network connection. Based on this, we give three corresponding methods to change the extracted features by morphing the malicious traffic packet to evade the detection. The structure of packet is shown in Fig. 5.

• *Method 1: Modifying length-related features.* The payload length of the network traffic packet can be modified by adding meaningless characters. Specifically, we choose to add the character 0xFF in this method. Similarly, an adversary can add meaningless characters to change the length of the malicious packet. It should be noted that the length of the modified payload

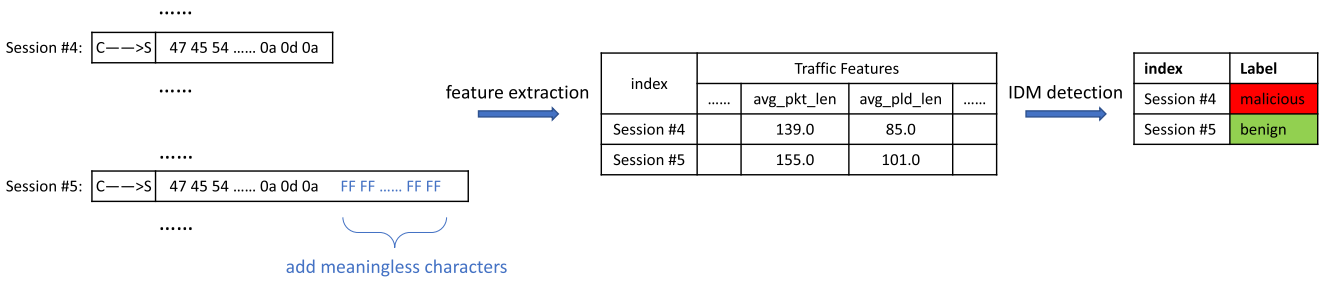


Fig. 3. Method 1: adding payload length. The length-related features are increased by injecting meaningless characters such as 0xFF into the original traffic packet.

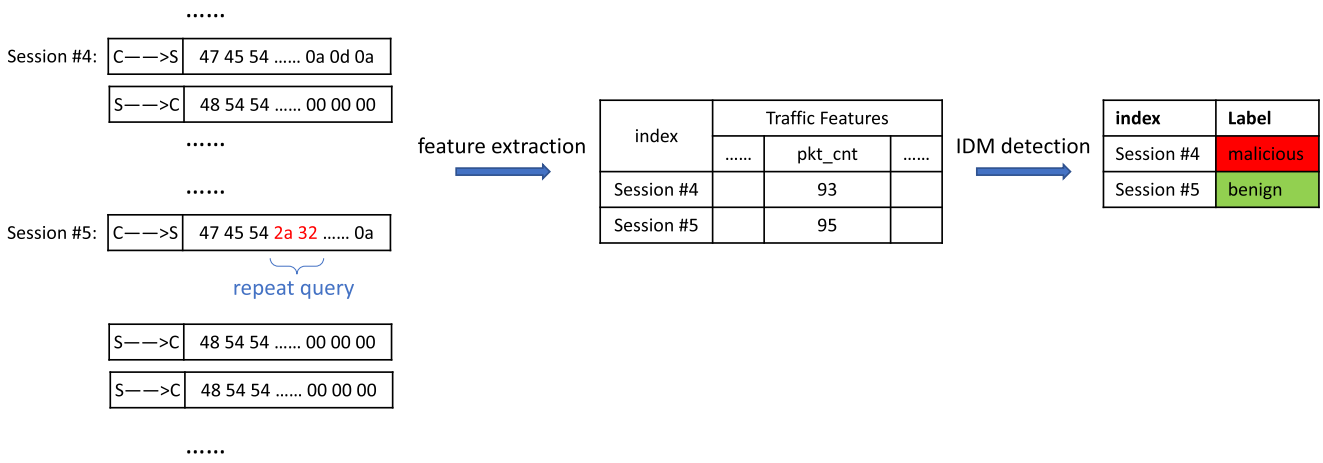


Fig. 4. Method 2: increasing packet numbers. The number of packets in one session can be increased by repeating the request multiple times. For example, adding characters 0x2a 0x32 (x2) into the request file packet, i.e., request files twice, yields twice response packets.

must be less than the maximum transmission payload size in a single network-layer transaction (e.g., 1460 bytes for IPv4 TCP protocol). Our experiments show that adding up to 300 0xFF characters can change the length-related features of the traffic flow to be detected, and thus causing the flow to be misclassified by the classification model. Fig. 3 shows the process of method 1.

• *Method 2: Increasing the number of packets.* The number of packets contained in one complete session is often applied as an important feature in the field of network classification. An adversary can try to change this feature by splitting the requested content into more parts or by repeating some certain content to send, which is shown in Fig. 4. Note that the timestamp and sequence number of the modified packets are in accordance with the normal distribution, which will not be filtered out as the retransmission packet. This method is effective for classifiers that rely on features related to the number of packets. Fig. 4 shows the process of method 2.

• *Method 3: Modifying time-related features.* A considerable fraction of flow features are time-related, so the adversary can control the duration of the entire session or the interval between packets to change these features, e.g., by waiting an extra time before sending packets, to evade the detection, which is shown in Fig. 6. It should be noted that a long interval will lead to a connection timeout. Our experiments show that for some simple

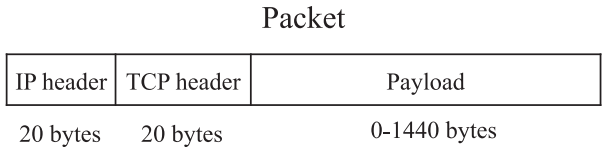


Fig. 5. The packet structure. The length-related modifications are mainly in the payload.

classifiers, it is effective for the adversary to add only 0.001ms of delay before each packet is sent. Fig. 6 shows the process of method 3.

Note that though the adversary does not know the feature dimensions used by the target model, when the traffic example is perturbed by the three methods mentioned above, the features extracted by the target model will also change. We also verify this statement in our experiments. Meanwhile, these perturbations are designed in terms of the overall statistical features of the traffic flow rather than individual key packets of malicious traffic or fields of its payload. They are concentrated on three macroscopic dimensions of length, time, and number, and will not affect the functionality of the packet. Therefore, in the case where the perturbations all follow the network protocol specification, the original maliciousness of the modified traffic sample will not be

Fig. 6. Method 3: Changing interval time. The time-related features are increased by adding a delay, e.g., 0.001 ms, before each packet is sent from the client.

lost. Based on this principle, many features of other dimensions can also work after being modified, which will not be discussed due to space limitations.

2) Analysis of Adversarial Traffic Examples: Previous research on malicious sample construction is mainly studied in two directions. The first one is beyond the knowledge of the classifier. The classification model does not use any malicious sample for iterative training, resulting in the classifier being unable to determine the sample based on the "experience", which creates a blind spot. The second one is the classification error. Malicious samples are misjudged due to errors in the classifier's decision boundary, which can be exploited by the adversary to evade detection. Essentially, both methods are expected to let the target model misclassify malicious traffic samples to achieve the evasion attack.

In the transformation from traffic to features, adversaries can maliciously tamper with samples. Although trivial changes in individual features may play a limited role, adding them together can cause large changes in the output of ML-based IDS. In fact, the more accurate classifiers tend to require more features, and thus it is easier to craft samples to evade their detection. The methods proposed in Section III-C1 follow all three directions and aim to change the features of traffic by morphing packets to evade detection.

D. Local Transfer Attack

Considering that random morphing could yield some successful examples yet bring massive computation cost [25], here we formulate the morphing procedure as an optimization problem to present the specific modification direction with less cost. Adversaries can perform the adversarial example on the white-box local substitute model to find the modified traffic which is misclassified by the target model because the transferability holds between the local model and the target model. However, existing adversarial example generation schemes do not guarantee the protocol functionality of the generated traffic since the traffic packet is constrained by network protocols and cannot be modified or generated in a manner like the image. Therefore, we directly morph the real-world malicious traffic packet using three methods in Section III-C1, which will not reduce the maliciousness of the traffic.

We first present the optimization problem used to craft adversarial features on the local substitute model. Then we show how to get the adversarial traffic example from the adversarial features by three methods in Section III-C1.

1) Optimization Problem: Given a legitimate input $\boldsymbol{x}$ and its ground truth $y = f_{oracle}(\boldsymbol{x})$, the goal of local transfer attack is to find an instance $x^\star$ satisfying $f_{oracle}(\boldsymbol{x}^\star) \neq y$ by morphing $\boldsymbol{x}$ with perturbation δ, i.e., $\boldsymbol{x}^\star = \boldsymbol{x} + \delta$. Formally, we train k local substitute models denoted as $\sum_{i=1}^{k} \alpha_i f_i(\boldsymbol{x})$, where α_i are the ensemble weights and $\sum_{i=1}^{k} \alpha_i = 1$. We can approximate the local transfer attack to solve the following optimization problem:

$$\underset{\boldsymbol{x}\star}{\mathbf{argmin}} -L\left(\left(\sum_{i=1}^{k} \alpha_i f_i(\boldsymbol{x}^\star)\right), y\right) + \lambda d(\boldsymbol{x}, \boldsymbol{x}^\star), \quad (2)$$

where L is to measure the distance between the prediction from local models and the ground truth from the target oracle, d is a metric to quantify the distortion between two input examples, λ is a constant which is often set empirically. Besides, L we choose here is

$$L(\boldsymbol{x}_i, \boldsymbol{x}_j) = \log(1 - \boldsymbol{x}_i \cdot \boldsymbol{x}_j), \quad (3)$$

which is shown effective in many previous works [12], [24]. The distortion d between the original example and the adversarial example are calculated by

$$d\left(\boldsymbol{x}^\star, \boldsymbol{x}\right) = \sqrt{\sum_{i} \left(\boldsymbol{x}^{(i)\star} - \boldsymbol{x}^{(i)}\right)^2 / N}, \quad (4)$$

where $\boldsymbol{x}^{(i)}$ denotes the i-th feature value of the original example $\boldsymbol{x}$, $\boldsymbol{x}^{(i)\star}$ denotes the one of the adversarial example, and N denotes the number of the feature dimensionality. This is used to measure the modification strength in adversarial examples.

In this work, adversaries aim to evade the ML-based IDS detection, thus only considering the type of non-targeted attack is enough in this scenario. The targeted attack can be derived

TABLE I
TRAFFIC DATASETS

Dataset	Size	Flow	Features
Normal Network Traffic	14,547MB	31,727,824	40
Network Flooding Attack	354MB	7,926,463	40
Botnet Malware Attack	349MB	7,806,225	40
Web Application Attack	378MB	5,131,840	40
Brute Force Attack	373MB	8,355,701	40
Total	15.63GB	60,948,053	40

similarly by replacing the y of the loss function in (2) with target label $y^\star$.

2) Solution: The optimization problem (2) can be solved by the iteration training. In the iteration procedure, the adversary fixes parameters of the local model and continuously adjusts the adversarial feature $\boldsymbol{x}^\star$ to let the local model misclassify. The direction of adjustment keeps the same as the direction of gradient descent. The adjustment directly acts on $\boldsymbol{x}^\star$. This process can also directly use some existing adversarial example algorithms, such as PGD [26], to find the final adversarial feature. When reaching the maximum number of iterations set previously or evading the model's detection, the $\boldsymbol{x}^\star$ is output. Then, the adversary perturbs the traffic t to find the corresponding $\boldsymbol{x}^\star$, which is shown as

$$\boldsymbol{x}^\star = \mathrm{E}(t + p), \tag{5}$$

where E denotes the feature extraction function, t is the traffic example to be morphed, and p is added perturbations. The perturbation methods in Section III-C1 change linearly. Consequently, $t' = t + p$ is the constructed adversarial traffic sample. Note that the adversary can flexibly select some or all feature dimensions of $\boldsymbol{x}^\star$ to participate in perturbation according to the feasibility of dimensional modification.

IV. EVALUATION

In this section, we reproduce malicious traffic identification models and use our framework to evade the detection of ML-based IDSs.

A. Building Target Model

1) Datasets: The malicious traffic detection model monitors traffic to and from all devices in real-time for malicious activity or policy violations We launch a variety of attacks, such as Denial of Service attack, Man-in-the-Middle attack in LAN. These attacks are the major network threat currently. We also collect normal traffic including HTTP, FTP, chat, etc. Then, we collect and classify these attack flows which are intercepted from multiple collection stations during different periods, as shown in Table I.

Furthermore, we reconstruct the automatic traffic filtering and feature extraction algorithm. Forty representative features, which are very effective and commonly used in traffic classification [27], [28], [29], are selected to form a set of feature vectors for the following experiment. The details of features can be found in Table II. Note that adversaries have no knowledge of these models, including model algorithms and traffic features.

TABLE II
THE EXTRACTED FEATURES USED FOR ML-BASED IDS TRAINING

40 Features used by ML-based IDSs					
(a)	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts
	Fwd IAT Tot	Fwd IAT Std	Fwd IAT Max	Bwd IAT Mean	Bwd IAT Std
(b)	Pkt Len Max	Pkt Len Std	FIN Flag Cnt	SYN Flag Cnt	ACK Flag Cnt
	Subflow Bwd Byts	Init Bwd Win Byts	Fwd Act Data Pkts	Active Std	Active Max
(c)	Bwd Pkt Len Mean	Bwd Pkt Len Std	Flow Byts/s	Fwd Pkt Len Std	Bwd Pkt Len Max
	Bwd PSH Flags	Fwd Header Len	Bwd Header Len	Bwd IAT Max	Bwd IAT Min
(d)	Subflow Fwd Pkts	Subflow Fwd Byts	Subflow Bwd Pkts	Down/Up Ratio	Bwd Seg Size Avg
	Flow IAT Std	Fwd Pkts/s	Bwd Pkts/s	Idle Std	Flow Pkts/s

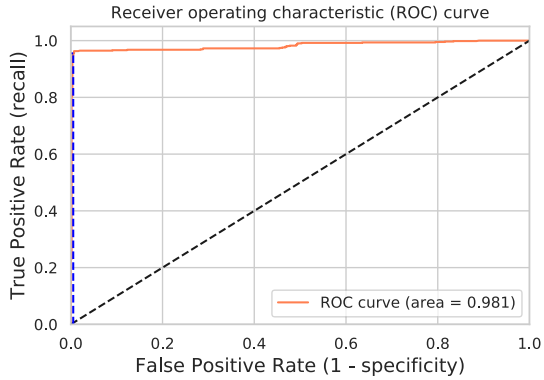


Fig. 7. ROC (Receiver Operating Characteristic) curve of Logistic Regression based IDS.

2) Model Algorithm: Here we elaborate on the details of ML-based IDS training. Seven common machine learning models are implemented as ML-based IDSs using the scikit-learn framework, which is very mature and widely used in real-world scenarios nowadays [30]. We first show the performance of ML-based IDSs. We split the dataset into 70% for the training set and 30% for the testing set. Each test is run 10 times with randomly shuffled datasets and the results are averaged. For example, the LR model uses the BFGS algorithm for convergence and iterates up to 50,000 times. As a result, the average overall accuracy of the LR model is 98.10%, consuming 52.4s on average. The average log loss of the LR model is 0.1070. The AUC of the LR model is 0.9810. Fig. 7 shows the ROC curve of the LR model.

B. Extracting Target model

1) Synthetic Queries: We reproduce three synthesis queries in Section III-A. Let n denotes the number of samples in the target model's training dataset, we set the budget of synthetic query from $0.2n$ to $2.0n$ to test its performance. The budget denotes the number of training examples needed to construct the local substitute model, which is a very important metric to evaluate the effectiveness of our proposed method. For query 3 (dataset migration), CIC-IDS-2017 datasets are used to request the target model. For query 1 (linear query matrix), the adversary constructs traffic examples whose extracted features are extreme values. For example, when the payload of the constructed example is empty, its length-related features are all minimum.

We also choose the most typical linear model Logistic Regression as the target model to verify our statement in Section III-A1. We divide the 40 features into 4 groups (shown in Table II) according to their importance and use each group separately to construct the linear query matrix to query the target LR model. The results are shown in Fig. 8. Although the importance of

TABLE III
THE ACCURACY OF DIFFERENT LOCAL SUBSTITUTE MODELS EXTRACTED FROM THE TARGET ML-BASED IDS USING DIFFERENT ALGORITHMS

Target ML-based IDS	Accuracy	Budget	Logistic Regression	SVM	Naive Bayes	Decision Tree	Random Forest	xgBoost	Neural Network	Ensemble Model
Logistic Regression	0.9810	0.2	0.4116	0.3916	0.5800	0.6866	0.6500	0.7450	0.4016	0.5016
		0.4	0.8651	0.8577	0.8714	0.7546	0.8577	0.8761	0.8651	0.8872
		0.6	0.8809	0.8828	0.8835	0.7885	0.8843	0.8843	0.8835	**0.8901**
SVM	0.9816	0.2	0.6251	0.6553	0.5267	0.7214	0.6338	0.7712	0.5264	0.5465
		0.4	0.8541	0.8794	0.8619	0.8541	0.8742	0.8356	0.8691	0.8894
		0.6	0.8629	0.8938	0.8857	0.8629	0.8887	0.8835	0.8843	**0.8916**
Naive Bayes	0.9253	0.2	0.7184	0.6671	0.7514	0.7314	0.6968	0.7631	0.3946	0.5534
		0.4	0.8372	0.8645	0.8571	0.7791	0.8349	0.8342	0.8471	0.8850
		0.6	0.8784	0.8742	0.8835	0.8179	0.8496	0.8496	0.8592	**0.8909**
Decision Tree	0.9005	0.2	0.3616	0.3433	0.6250	0.7366	0.6483	0.6156	0.3733	0.5116
		0.4	0.6286	0.5453	0.7061	0.8268	0.8759	0.6835	0.5458	0.7936
		0.6	0.7811	0.7731	0.7347	0.8592	0.8813	0.7111	0.5607	**0.8909**
Random Forest	0.9614	0.2	0.8087	0.6281	0.7617	0.7480	0.7579	0.7981	0.7963	0.4637
		0.4	0.8637	0.8701	0.8635	0.8372	0.8731	0.8646	0.8406	0.8903
		0.6	0.8695	0.8798	0.8703	0.8431	0.8865	0.8651	0.8496	**0.8998**
xgBoost	0.9688	0.2	0.4635	0.4431	0.6012	0.7111	0.6873	0.7681	0.5879	0.4542
		0.4	0.8865	0.8879	0.8781	0.7351	0.8765	0.8504	0.8048	0.8879
		0.6	0.8872	0.8879	0.8813	0.7435	0.8843	0.8879	0.8408	**0.8891**
Neural Network	0.9601	0.2	0.1444	0.1121	0.3506	0.2351	0.5777	0.1112	0.4532	0.5743
		0.4	0.7748	0.7191	0.6846	0.7943	0.7452	0.7191	0.7251	0.7737
		0.6	0.8066	0.8633	0.7963	0.8361	0.8550	0.8631	0.8834	**0.8916**

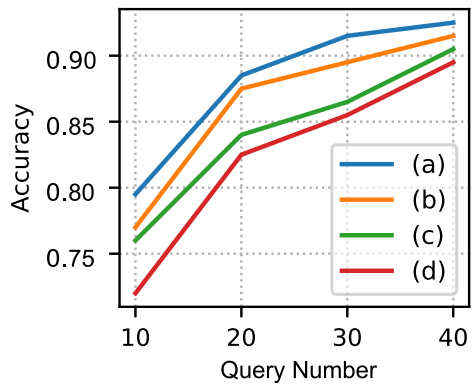


Fig. 8. Construct 4 kinds of linear query matrixes we proposed based on 4 feature groups in Table II to extract target logistic regression based IDS. The more the number of queries, the higher the accuracy of the extracted LR model.

group (d) features is the lowest, from the results we can see that our method is still able to use (d) to achieve good extraction results when the number of queries is sufficient, i.e., up to 40.

Table III summarizes the the query experiment results. All results are averaged over 10 runs to obtain more stable results. Testing on different target models, we find query 1 is effective for the linear model and obtains the exact parameters of the target model with the minimum number of queries, which is consistent with the effect of model extraction attack in [31]. However, query 1 works poorly for nonlinear models, the results obtained from its queries mainly focus on one specific category. For example, all the queries from query 1 are judged as Network Flooding Attack type by the SVM model with confidence above 0.89. Query 2 and query 3 both work well. When the query budget is 0.2n, the accuracy of the extracted local model is basically usable. When the budget reaches 0.6n, the accuracy of most local models is above 0.85. Continuing to increase the budget, the effect is not obvious, so we do not continue to list the results due to the space limitation.

2) Ensemble Learning: We let the adversary use these seven algorithms respectively to retrain the model on the synthetic dataset. The accuracy of local substitute models on synthetic datasets is also shown in Table III.

The ensemble model is trained locally using autoML technology [23]. The ensemble learning is able to make full use of the synthetic dataset, and generate models with the highest accuracy, which actively demonstrates that the extracted model fits well with the decision boundary of the target model and is more conducive to transfer attack. At the same time, the random forest model, one of the representatives of ensemble learning in the results, works better than a single decision tree model, which also shows that ensemble learning works well. The only disadvantage of the ensemble model is the attack overhead and complexity because an optimal combination of algorithms is needed to search, but it is still worthwhile as the transfer success rates are improved.

C. Crafting Adversarial Example

The specific model combination in the local ensemble is determined by the autoML default selector. Since the ensemble model provides a higher transfer rate against target models than one single model, to be consistent, we use this configuration in all our experiments attacking target ML-based IDSs.

100 unseen traffic examples are randomly sampled from each of the 4 malicious classes for 400 total examples and then are used to find adversarial examples. White-box PGD attacks are performed on the ensemble loss with 100 iterations to provide the direction of estimated gradients for the example perturbing. We choose $\epsilon = 0.3$ following the same setting in [26]. Finally, these examples are tested on the black-box target model to measure

TABLE IV
THE PERFORMANCE OF ADVERSARIAL MALICIOUS TRAFFIC EXAMPLES ON THE LOCAL SUBSTITUTE MODEL AND THE TARGET ML-BASED IDS

Target ML-based IDS	Method 1: Add characters			Method 2: Increase packets			Method 3: Change time		
	Success Rate	Transfer Rate	Evade Rate	Success Rate	Transfer Rate	Evade Rate	Success Rate	Transfer Rate	Evade Rate
Logistic Regression	0.8327	0.7133	0.5939	0.8655	0.7333	0.6346	0.8317	0.7119	0.5920
SVM	0.8095	0.6800	0.5504	0.9067	0.8841	0.8016	0.5714	0.4457	0.2546
Naive Bayes	0.8317	0.7119	0.5920	0.8637	0.7769	0.6710	0.6667	0.5041	0.3360
Decision Tree	0.7879	0.6521	0.5137	0.7134	0.5436	0.4265	0.3589	0.2802	0.1005
Random Forest	0.6364	0.4667	0.2970	0.7846	0.6146	0.4384	0.2104	0.0861	0.0181
xgBoost	0.8317	0.7119	0.5920	0.8411	0.7949	0.6685	0.4685	0.2597	0.1216
Neural Network	0.8455	0.7967	0.6736	0.8769	0.7158	0.6276	0.5871	0.4899	0.2876

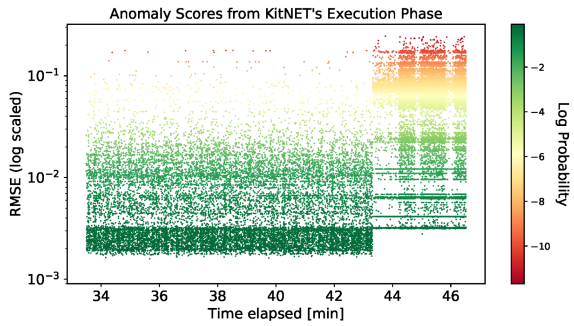
the evading performance. Besides, for method 1, the maximum length of added characters is set to 300, and the average distortion is 2.3818. For method 2, the maximum number of added traffic packets is set to half of the packet number of one session and the average distortion is 2.4719. For method 3, the increase interval is set to 0.001ms and the average distortion is 0.9136.

1) Metrics: Three metrics, $success\ rate$, $transfer\ rate$ and $evade\ rate$, are used to evaluate the effect of crafting example. Let $\mathcal{P}$ refers to the number of examples provided by the adversary, $\mathcal{L}$ refers to the number of examples misclassified by the local substitute model, and $\mathcal{T}$ refers to the number of examples misclassified by the target model. Note $\mathcal{P} = 400$. We denote $success\ rate = \mathcal{L}/\mathcal{P}$, which is used to evaluate the effect of the adversarial example against the local model. $transfer\ rate = \mathcal{T}/\mathcal{L}$, which is used to evaluate the effect of the adversarial example against the target model. $evade\ rate = \mathcal{T}/\mathcal{P}$, which is used to evaluate the performance of our framework and is our primary metric.

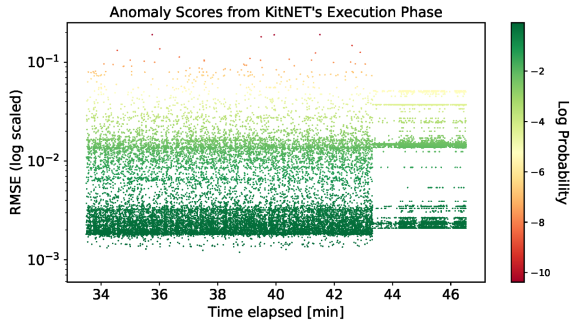
Table IV summarizes the evading experiment results. All results are averaged over 10 runs to obtain more stable results. From the results, the added perturbations allow the adversarial malicious traffic example to evade the detection of different target ML-based IDSs with at most 80% probability after test on the local substitute model. The perturbation added by method 3 is less than others, so the performance of method 3 is low, but its $evade\ rate$ can be improved by increasing the perturbation. In addition, the high $evade\ rate$ often comes with high $success\ rate$. The reason is that the better the classification effect of the model, the more dependent it is on the features. High $success\ rate$ indicates that the model has a poor classification effect on the disturbed samples, so the evasion performance of the framework is better. Besides, the results also show the generalization ability of the proposed attack, which works for different types of traffic classification models. The transfer performance is also consistent with the findings in [24].

D. Attacking Stronger DL-Based IDS

Previous evaluations are all based on various typical network ML-based IDSs, to illustrate the performance of our evasion attack more effectively, here we reproduced the state-of-the-art (SOTA) published network IDS Kitsune [4]. Kitsune is an unsupervised DL-based IDS, using an ensemble of autoencoders to differentiate between normal and abnormal network traffic samples. We train Kitsune by its published traffic datasets and its default parameters. In this work, Kitsune learns the normal traffic patterns and behaviors from 70,000 benign traffic instances. The test datasets consist of 15,000 benign examples and 15,000 malicious examples.



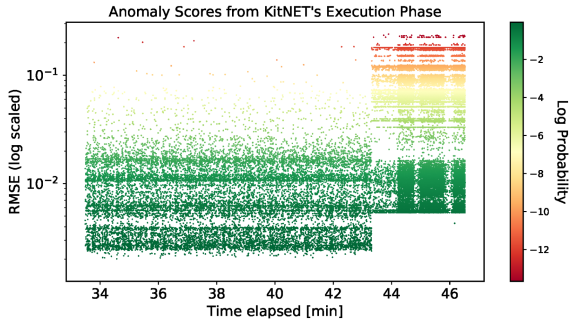
(a) The detection performance of the SOTA network DL-based IDS against the web application attack.



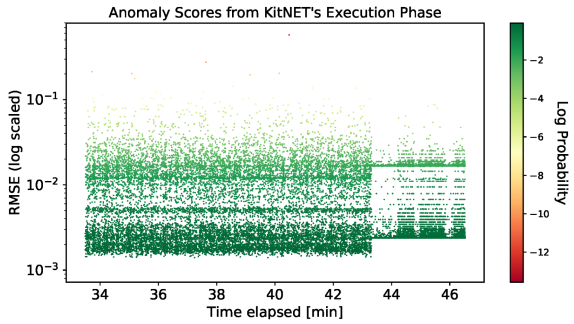
(b) The evasive effectiveness of adversarial web application attack traffic processed by our proposed framework.

Fig. 9. Against the web application attack, the detection performance of the SOTA network DL-based IDS. Each dot denotes one traffic sample to be detected. The test samples are input after 43min. A higher RMSE score reflects a higher anomaly, i.e., a malicious sample. The yellow and red dots indicate the sample is detected as the web application attack traffic packages with high anomalies.

Kitsune outputs the root mean squared errors (RMSE) score to indicate the degree of abnormality of the traffic example. Concretely, in this work, Kitsune is trained to fit the benign output score to log-normal distribution for a more obvious distinction as the same setting in [4]. The malicious example with a very low occurring probability will easily raise the alert. The anomaly scores for each traffic sample in test datasets are generated from KitNET (Kitsune's core algorithm) and shown by dots after 43min in Fig. 9 and 10. We also adjust the packet timestamp in chronological order for a better display effect.



(a) The detection performance of the SOTA network DL-based IDS against the DDoS attack.



(b) The evasive effectiveness of adversarial DDoS attack traffic processed by our proposed framework.

Fig. 10. Against the DDoS attack, the detection performance of the SOTA Network DL-based IDS. Each dot denotes one traffic sample to be detected. The test samples are input after 43min. A higher RMSE score reflects a higher anomaly, i.e., a malicious sample. The yellow and red dots indicate the sample is detected as the DDoS attack traffic packages with high anomalies.

The web application attack and DDoS attack are involved in this test. The results shown in Figs. 9(a) and 10(a) demonstrate the detection effectiveness of Kitsune. The yellow and red dots indicate the corresponding abnormal traffic examples, with a high probability of being malicious samples.

Then CIC-IDS2017 datasets are used to perform the model extraction attack and ensemble learning technology is used to train the local substitute models. We generate the adversarial web application attack and DDoS examples using our proposed framework and take 5,000 locally successful samples to test. The evasive effectiveness is shown in Figs. 9(b) and 10(b). For the DDoS attack, most of the adversarial examples generated by our framework evade the detection of the DL-based IDS. For the web application attack, there are still partial samples with a higher probability to be detected, as shown by the yellow dots in Fig. 9(b). These are mainly password brute force packages. Due to the need to repeatedly send packages for testing, the traffic features of modified examples are still far from the normal web application traffic packages, but the RMSE gap has been significantly shortened compared with that before modification.

E. Verifying Morphed Traffic

Though the attack effectiveness of evasive traffic example is ensured in our construction methods, since our morphing methods have not compromised the functionality of modified packets and made them conform to the protocol format, here we verify the attack effectiveness of the morphed attack traffic compared with the selected original attack traffic without morphing.

TABLE V
FUNCTIONAL VERIFICATION OF MORPHING MALICIOUS TRAFFIC. THE COMPARISON RESULTS DEMONSTRATE THE TRAFFIC MORPHED BY OUR METHODS STILL RESULTS IN SUCCESSFUL ATTACKS

Attack	Effectiveness Measurement	Original	Morphed
Network Flooding	Network bandwidth occupation	456Mbps	348Mbps
Botnet Malware	Number of device scans	1000	975
Web Application	Number of website attacks	1000	846
Brute Force	Number of password attempts	1000	1000

The results are shown in Table V. Different attacks have their corresponding attack effectiveness measurements. From the comparison, we can see that our methods can preserve the malicious functionality of the morphed traffic. The reason for a few failed cases is that the increase of feature value in the time dimension leads to the reduction of attack efficiency, which is also consistent with the finding in Section IV-C that method 3 modifies the time-related features with a lower success rate than the other two methods.

V. COUNTERMEASURES

The attack proposed in this paper can be defended from two perspectives. One is to defend against the model extraction attack, and the other is to improve model robustness. There are also some other defenses such as gradient masking, which prevents the adversary from getting useful gradients to construct the adversarial examples directly. However, such defenses are not effective for the black-box attack used in this work, since the adversary can still obtain the gradient direction from the local substitute model based on the transferability of the adversarial examples.

A. Defending Against Model Extraction Attack

Transfer attacks rely on the local substitute model extracted from the target model. Thus, the defender can protect the model from being extracted. However, directly restricting the service of the target model will affect the benefit of the service provider. It is also ineffective against collusion attacks. Besides, note that model extraction attack cannot be fully defended, the defender can only consider increasing the cost of model extraction or reducing the accuracy of the extracted model. Currently, existing defense schemes are mainly based on perturbation and monitoring, both of which have made good progress. The scheme in [32] that combines the above two schemes is also worthy to be adopted by defenders. Here we do not repeat the verification of this kind of defense.

B. Improving Model Robustness

In addition to improving the model's ability to detect adversarial example inputs, it is also feasible to improve the robustness of the model, which is also known as adversarial learning. This kind of method puts malicious samples into the model training dataset in advance. The model is exposed to as many adversarial examples as possible during the training phase to

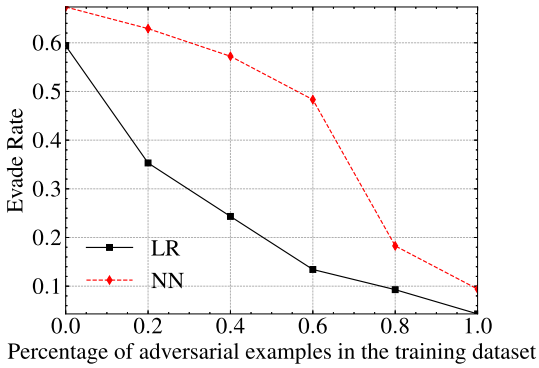


Fig. 11. The Evade Rate of Logistic Regression (LR) and Neural Network (NN) models trained on datasets containing adversarial samples. The lower the evade rate, the better the robustness of the model after adversarial training.

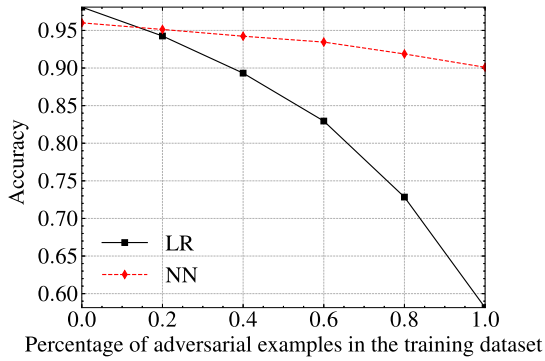


Fig. 12. The use Accuracy of Logistic Regression (LR) and Neural Network (NN) models trained on datasets containing adversarial examples. The higher the accuracy, the better the model performance of the model, and the less affected after adversarial training.

correct the learned decision boundary. This scheme significantly improves the robustness of the model and effectively counteracts adversarial sample attacks.

Here we evaluate the effectiveness of adversarial training against our proposed evasion strategy. First, we add adversarial traffic examples into the training datasets of the target model. Examples are generated by our strategy and used to build a robust classifier in the ML-based IDS. Then, we repeat our evasion strategy against the robust model, comparing it with the normal model without adversarial examples in the training dataset. Due to the space, we select two representative models LR and NN to illustrate the robustness improvement. The results of other models are quite similar.

From Fig. 11, we can see that adversarial training is effective to improve the robustness of ML-based IDSs and defense against our proposed evasion attack. The reason is that adversarial examples make the decision boundary of models more complicated in a robust way. For the model, more new examples increase the model's "knowledge" and reduce the effective space of adversarial example's features. From another angle, our work can also be used to generate adversarial traffic examples and feed them into the training dataset of the deployed IDS model to improve model robustness.

Another interesting observation is that we find adversarial training changes models' feature importance distribution, which decreases the classification accuracy of normal traffic, shown in Fig. 12. We verify this conjecture by the feature weight of models. The results show in Fig. 13. Models' features after adversarially trained are concentrated in key features, and the feature distribution of models whose training set without adversarial examples is relatively noisy. For the limited capacity model like LR, adversarial training improve the robustness against adversarial traffic examples at the expense of accuracy on normal examples, which is a disadvantage of the method. Another disadvantage is that when the model encounters the adversarial example not in the adversarial training datasets, it still cannot perform effective defense.

C. Adjust Feature Dimension of Model

Adversarial examples are related to feature dimensions in models. Feature selection and reduction [33], [34] can be used to find dimensions that are more susceptible to perturbation, then reduce corresponding feature weights or directly remove vulnerable features to prevent the evasion attack. This kind of method can limit the generation of adversarial examples to a certain extent as it is more difficult for adversaries with fewer available features and more perturbation needed to add. However, the work in [35] shows that limiting the number of features that the adversary can control does not completely prevent this type of attack, even if 5 random features can give adversarial samples a 50% success rate. Besides, reducing some vulnerable feature dimensions that may also work for normal examples will result in a decrease in accuracy, which is unacceptable for tasks requiring extremely high classification accuracy. Here we also do not repeat the verification of this kind of defense.

VI. Discussion

The experiment results demonstrate that an adversary using our framework has a very high probability to evade various ML-based IDSs. The results are in line with our expectations. The black-box scenario considered throughout this work is also very realistic, which fully reflects the threat of this security issue. In addition, ML-based malicious traffic classification models are heavily dependent on the traffic sample features, so we make the malicious traffic samples plausible by morphing them to change their features. Finally, the modified sample can evade the detection of ML-based IDSs while maintaining the original maliciousness.

Our work is a specific application of adversarial examples in the network traffic classification scenario. Almost all previous works focus on the domain of image classification. Unlike images, which inherently contain explicit features, network traffic example follows a certain protocol format and its features are statistical. Therefore, these attack schemes for images, such as [16], cannot be used directly in the traffic scenario. In addition, the current query generation schemes for images, such as [36], are also ineffectual on network traffic. The reason is that network traffic needs to comply with protocol standards without losing functionality, otherwise, it will be filtered by the ML-based IDS directly. To solve the above problem, we devise a specialized generation scheme for synthetic queries with low cost and good

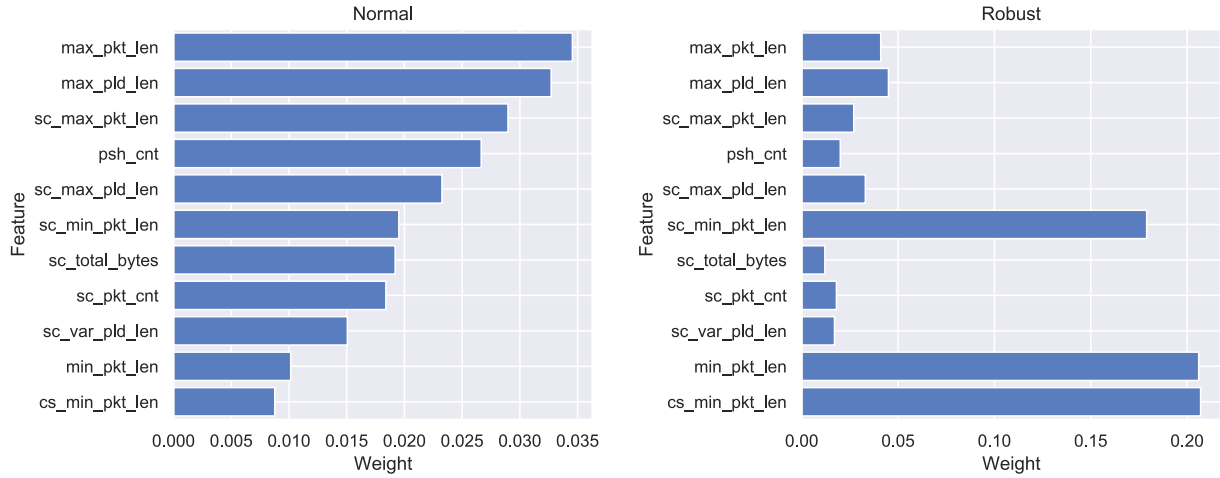


Fig. 13. The distribution change of top 11 feature weight in normal model and robust model after adversarial training.

TABLE VI
THE COMPARISON RESULTS OF THE REALISTIC CONDITIONS MET OR NOT

Works	Require **NO** Knowledge of				Traffic Sample		
	Extracted Feature	Model's Parameter	Model's Algorithm	Confidence of Output	Modification Dimension	Evasion Effectiveness	Protocol Functionality
[39]	✘	✘	✘	✘	feature	✔	✘
[40]	✘	✔	✔	✘	feature	✔	✘
[41]	✘	✔	✔	✔	feature	✔	✘
[8]	✘	✔	✔	✘	packet	✔	✔
[15]	✔	✔	✔	✔	packet	✘	✔
[42]	✔	✔	✔	✔	packet	✘	✔
Our work	✔	✔	✔	✔	packet	✔	✔

effect of training substitute model, which also improves the efficiency of traffic classification model extraction. The ensemble learning we used in model extraction does work better than a single model, which is consistent with the findings in [24].

• *Limitations.* There are two limitations of this work. First, traditional payload-based IDSs are not considered in this work. This kind of IDSs' rules are usually provided by protocol analysis and deep packet inspection (DPI). The main consideration in this paper is the ML-based solution. Second, a high false-positive model or white-list model will significantly weaken the attack. However, this type of model is less commonly used in reality.

• *Future Work.* In future work, we plan to achieve more accurate and lower cost of model extraction and evasion detection and apply the adversarial example to more problems in the field of traffic besides the ML-based IDS. Although transfer attack is easy to use and effective, there are some more advanced attacks [37], [38], which have lower attack cost and are less likely to be detected, so these attacks will later be introduced into our framework to improve the efficiency of the attack. Moreover, the work of this paper belongs to the untargeted attack, which aims at expecting malicious traffic to be considered benign by the target model and evade detection. For targeted attacks, its goal is to make the model output a specific target class, rather than just misclassification. Targeted attacks have more application scenarios, but their transfer loss is high, which is more challenging.

VII. RELATED WORK

Machine learning is widely used to solve security issues in the field of network traffic. As the core component of network security systems, IDSs also adopt machine learning classification models to monitor and discover malicious traffic in recent years. However, many evasion attacks [43], [44] against IDS do not consider ML models. As a result, this work focuses on constructing adversarial traffic examples to evade ML-based IDSs.

Some works [8], [39], [40], [45] assume that the adversary knows more or less information about the target model, such as similar training datasets, model structure, or confidence scores of the results. To be more realistic, this work strictly restricts the black-box experimental scenario, where the adversary only obtains the label when accessing the model.

Some works do not consider the feasibility and cost of adversarial traffic samples. [41] directly modify the feature of traffic example, which is not feasible in practice. [15], [42], [46] randomly modify or/and obfuscate the traffic sample to evade the detection, resulting in low attack efficiency and high cost. Besides, two works also adopt the transfer attack to evade the detector. Papernot et al. [16] improves the synthetic query by using reservoir sampling [47] and Jacobian-based dataset augmentation techniques. Li et al. [36] uses an active learning strategy to further reduce the number of queries. However, these schemes cannot be applied to this problem as neither their synthetic query nor adversarial example construction scheme

is suitable for network traffic whose features are statistics. This paper proposes the traffic example crafting method and performs the transfer attack to address these two limitations. We also make a comparison of our work and previous works in Table VI, which shows that our work meet the most realistic conditions and is more practical in the real environment.

There are also some works to improve the transferability of adversarial samples. Liu et al. [24] propose novel ensemble-based approaches to improve the accuracy of transfer attacks. Dong et al. [48] propose momentum-based iterative algorithms to boost adversarial attacks. They also propose a translation-invariant attack method to generate more transferable adversarial examples in [49]. Xie et al. [50] improve the transferability of adversarial examples by applying random transformations to inputs at each iteration. This work mainly focuses on how to successfully apply adversarial sample techniques into traffic classification scenarios, these improvements can be taken into account in future work for specific traffic problems.

VIII. Conclusion

In this paper, we investigate a restricted real label-only black-box scenario for the adversary to evade the detection of intrusion detection. The adversarial example technique is introduced into the field of malicious traffic identification to successfully implement the evasion attack. At the same time, three methods for constructing traffic query samples and three methods for generating adversarial malicious traffic examples are given to strengthen the extraction and evasion effect. Ensemble learning is also introduced to improve the extraction effect. Besides, although the main evaluations are on intrusion detection tasks, the proposed framework has wider applications to other learning-based systems.

Acknowledgments

We thank the editor and all the anonymous reviewers for their valuable guidance and constructive feedback.

References

[1] J.-M. Flaus and J. Georgakis, "Machine learning based intrusion detection approaches for industrial IoT control systems: A review," in *Proc. Int. Conf. Ind. Internet Things Smart Manuf.*, Sep. 2018.

[2] A.-R. Sadeghi, C. Wachsmann, and M. Waidner, "Security and privacy challenges in industrial internet of things," in *Proc. 52nd ACM/EDAC/IEEE Des. Automat. Conf.*, 2015, pp. 1–6.

[3] S. Gulghane, V. Shingate, S. Bondgulwar, G. Awari, and P. Sagar, "A survey on intrusion detection system using machine learning algorithms," in *Proc. Int. Conf. Innov. Data Commun. Technol. Appl.*, 2019, pp. 670–675.

[4] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA: The Internet Society, Feb. 18-21, 2018. [Online]. Available: https://dblp.org/rec/conf/ndss/MirskyDES18.bib

[5] F. Hachmi, K. Boujenfa, and M. Limam, "Enhancing the accuracy of intrusion detection systems by reducing the rates of false positives and false negatives through multi-objective optimization," *J. Netw. Syst. Manage.*, vol. 27, no. 1, pp. 93–120, 2019.

[6] L. Huang, A. D. Joseph, B. Nelson, B. I. Rubinstein, and J. D. Tygar, "Adversarial machine learning," in *Proc. 4th ACM Workshop Secur. Artif. Intell.*, 2011, pp. 43–58.

[7] M. A. Ayub, W. A. Johnson, D. A. Talbert, and A. Siraj, "Model evasion attack on intrusion detection systems using adversarial machine learning," in *Proc. IEEE 54th Annu. Conf. Inf. Sci. Syst.*, 2020, pp. 1–6.

[8] M. J. Hashemi, G. Cusack, and E. Keller, "Towards evaluation of NIDSs in adversarial setting," in *Proc. 3rd ACM CoNEXT Workshop Big DAta, Mach. Learn. Artif. Intell. Data Commun. Netw.*, 2019, pp. 14–21.

[9] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter, "Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1528–1540.

[10] N. Šrndić and P. Laskov, "Practical evasion of a learning-based classifier: A case study," in *Proc. IEEE Symp. Secur. Privacy*, 2014, pp. 197–211.

[11] W. Xu, Y. Qi, and D. Evans, "Automatically evading classifiers: A case study on PDF malware classifiers," in *Proc. 23rd Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA: The Internet Society, Feb. 21-24, 2016. [Online]. Available: http://wp.internetsociety.org/ndss/wpcontent/uploads/sites/25/2017/09/automatically-evadingclassifiers.pdf

[12] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 39–57.

[13] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "Textbugger: Generating adversarial text against real-world applications," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA: The Internet Society, Feb. 24-27, 2019. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/textbuggergenerating-adversarial-text-against-real-world-applications/

[14] H. Dang, Y. Huang, and E.-C. Chang, "Evading classifiers by morphing in the dark," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 119–133.

[15] I. Homoliak, M. Teknos, M. Ochoa, D. Breitenbacher, S. Hosseini, and P. Hanacek, "Improving network intrusion detection classifiers by non-payload-based exploit-independent obfuscations: An adversarial approach," 2018, *arXiv:1805.02684*.

[16] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017, pp. 506–519.

[17] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 686–728, First Quarter 2018.

[18] S. Chen, N. Carlini, and D. Wagner, "Stateful detection of black-box adversarial attacks," in *Proc. 1st ACM Workshop Secur. Privacy Artif. Intell.*, 2020, pp. 30–39.

[19] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.

[20] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.

[21] M. W. Craven and J. W. Shavlik, "Extracting tree-structured representations of trained networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 1996, pp. 24–30.

[22] M. Kesarwani, B. Mukhoty, V. Arya, and S. Mehta, "Model extraction warning in MLaaS paradigm," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, 2018, pp. 371–380.

[23] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-sklearn 2.0: The next generation," 2020, *arXiv:2007.04074*.

[24] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," 2016, *arXiv:1611.02770*.

[25] W. Brendel, J. Rauber, and M. Bethge, "Decision-based adversarial attacks: Reliable attacks against black-box machine learning models," 2017, *arXiv:1712.04248*.

[26] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.

[27] H. Kim, K. C. Claffy, M. Fomenkov, D. Barman, M. Faloutsos, and K. Lee, "Internet traffic classification demystified: Myths, caveats, and the best practices," in *Proc. ACM CoNEXT Conf.*, 2008, pp. 1–12.

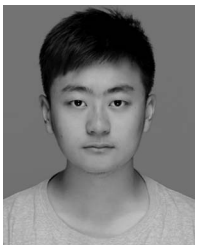
[28] W. Li, M. Canini, A. W. Moore, and R. Bolla, "Efficient application identification and the temporal and spatial stability of classification schema," *Comput. Netw.*, vol. 53, no. 6, pp. 790–809, 2009.

[29] H. Yan et al., "PGSM-DPI: Precisely guided signature matching of deep packet inspection for traffic analysis," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.

[30] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, Second Quarter 2018.

[31] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *Proc. 25th USENIX Secur. Symp.*, 2016, pp. 601–618.

[32] H. Yan, X. Li, H. Li, J. Li, W. Sun, and F. Li, "Monitoring-based differential privacy mechanism against query flooding-based model extraction attack," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2680–2694, Jul./Aug. 2022.
[33] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, no. Mar, pp. 1157–1182, 2003.
[34] D. Han et al., "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2632–2647, Aug. 2021.
[35] R. Sheatsley, N. Papernot, M. Weisman, G. Verma, and P. McDaniel, "Adversarial examples in constrained domains," 2020, *arXiv:2011.01183*.
[36] L. Pengcheng, J. Yi, and L. Zhang, "Query-efficient black-box attack by active learning," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 1200–1205.
[37] C.-C. Tu et al., "AutoZOOM: Autoencoder-based zeroth order optimization method for attacking black-box neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 742–749.
[38] A. Ilyas, L. Engstrom, A. Athalye, and J. Lin, "Black-box adversarial attacks with limited queries and information," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 2137–2146.
[39] Z. Wang, "Deep learning-based intrusion detection with adversaries," *IEEE Access*, vol. 6, pp. 38367–38384, 2018.
[40] X. Peng, W. Huang, and Z. Shi, "Adversarial attack against DoS intrusion detection: An improved boundary-based method," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell.*, 2019, pp. 1288–1295.
[41] G. Apruzzese, M. Colajanni, and M. Marchetti, "Evaluating the effectiveness of adversarial attacks against botnet detectors," in *Proc. IEEE 18th Int. Symp. Netw. Comput. Appl.*, 2019, pp. 1–8.
[42] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Citeseer, vol. 9, 2009.
[43] T.-H. Cheng, Y.-D. Lin, Y.-C. Lai, and P.-C. Lin, "Evasion techniques: Sneaking through your intrusion detection/prevention systems," *IEEE Commun. Surveys Tuts.*, vol. 14, no. 4, pp. 1011–1020, Fourth Quarter 2012.
[44] I. Corona, G. Giacinto, and F. Roli, "Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues," *Inf. Sci.*, vol. 239, pp. 201–225, 2013.
[45] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[46] E. Stinson and J. C. Mitchell, "Towards systematic evaluation of the evadability of bot/botnet detection methods," in *Proc. 2nd Conf. USENIX Workshop Offensive Technol.*, 2008, pp. 1–9.
[47] J. S. Vitter, "Random sampling with a reservoir," *ACM Trans. Math. Softw.*, vol. 11, no. 1, pp. 37–57, 1985.
[48] Y. Dong et al., "Boosting adversarial attacks with momentum," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 9185–9193.
[49] Y. Dong, T. Pang, H. Su, and J. Zhu, "Evading defenses to transferable adversarial examples by translation-invariant attacks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4312–4321.
[50] C. Xie et al., "Improving transferability of adversarial examples with input diversity," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2730–2739.



Haonan Yan received the bachelor's degree from Xidian University. He is currently working toward the PhD degree with the School of Cyber Engineering, Xidian University, Xi'an, China. His current research interests focus on machine learning security and network traffic.

Xiaoguang Li received the bachelor"s degree from Xidian University. He is currently working toward PhD degree with the School of Cyber Engineering, Xidian University, Xi'an, China. He visited the Department of Computer and Information Technology, Purdue University from 2019 to 2021. His current research interests focus on differential privacy.

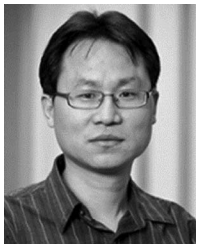
Wenjing Zhang is working toward the PhD degree with the School of Computer Science, University of Guelph, Canada. Her research interests include data privacy and machine learning.

Rui Wang received the BS degree in information security from Xidian University, in 2019. He is currenly working toward the master's degree with the School of Cyber Engineering, Xidian University. His research interests focus on tor and network traffic.

Hui Li (Member, IEEE) received the BSc degree from Fudan University, in 1990, and the MASc and PhD degrees from Xidian University, in 1993 and 1998, respectvely. Since June 2005, he has been a professor with the School of Cyber Engineering, Xidian University, Xi'an Shaanxi, China. His research interests are in the areas of cryptography, wireless network security, information theory, and network coding. He is a chair of ACM SIGSAC CHINA. He served as the technique committee chair or co-chair of several conferences. He has published more than 200 international academic research papers on information security and privacy preservation.

Xingwen Zhao received the BS and MS degrees from the School of Telecommunications Engineering of Xidian University, in 1999 and 2004 respectively, and the PhD degree from the School of Information Science and Technology of Sun Yat-sen University, in 2011. He is now an associate professor with the School of Cyber Engineering, Xidian University. He is also an editor of *International Journal of Technology in Computer Science & Engineering*. His research interests include machine learning (or artificial intelligent) based network security, multi-party data sharing, anonymous authentication, broadcast encryption, traitor tracing, key agreement.

Fenghua Li received the BS degree in computer software, and the MS and PhD degrees in computer systems architecture from Xidian University, Xi'an, China, in 1987, 1990, and 2009, respectively. He is currently a professor and a doctoral supervisor with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. He is also a doctoral supervisor with Xidian University and the University of Science and Technology of China. His current research interests include network security, system security, privacy computing, and trusted computing.



Xiaodong Lin (Fellow, IEEE) received the PhD degree in information engineering from the Beijing University of Posts and Telecommunications, China, and the PhD degree (with Outstanding Achievement in Graduate Studies Award) in electrical and computer engineering from the University of Waterloo, Canada. He is currently a professor with the School of Computer Science at the University of Guelph, Canada. His research interests include computer and network security, privacy protection, applied cryptography, computer forensics, and software security.